

Politechnika Warszawska
Wydział Elektryczny

Kierunek: Informatyka stosowana
Przedmiot: Języki i metody programowania 2

Dokumentacja Końcowa Projektu

Projekt C: Wyznaczanie współrzędnych węzłów dla wizualizacji
grafu planarnego

Autorzy:

Patryk Pawlak 343367
Krzysztof Dębski 343317

Warszawa, 19 kwietnia 2026

Spis treści

1	Wstęp	2
2	Architektura Systemu i Implementacja	2
2.1	Struktura Modułowa	2
2.2	Kluczowe Struktury Danych	3
2.3	Specyfikacja Wewnętrznego API (Wybrane funkcje)	3
3	Instrukcja Użytkownika (CLI)	3
3.1	Argumenty Uruchomieniowe	4
4	Format Danych	4
4.1	Plik Wejściowy (Tekstowy)	4
4.2	Plik Wyjściowy (Tekstowy)	5
5	Specyfikacja Mostka Binarnego (C-Java)	5
5.1	Struktura Rekordu Binarnego	5
6	Zaimplementowane Algorytmy Rozmieszczania	5
6.1	Algorytm Fruchtermana-Reingolda (Podejście siłowe, <code>-a fr</code>)	6
6.2	Algorytm Tutte’a (Twierdzenie o sprężynach, <code>-a wszystkie</code>)	6
7	Sytuacje Wyjątkowe i Obsługa Błędów	6
8	System Testowy i Walidacja Jakości	7
8.1	Środowisko i Metodologia	7
8.2	Raport z Wykonania Zestawu Testowego	8
8.3	Gwarancja braku wycieków (Valgrind)	8
9	Wnioski	8

1 Wstęp

Niniejszy dokument stanowi pełną specyfikację końcową modułu obliczeniowego **GraphVisualizer (C Core)**, realizowanego w ramach przedmiotu Języki i metody programowania 2.

Celem aplikacji jest automatyczne wyznaczanie współrzędnych (x, y) wierzchołków grafu planarnego w celu jego czytelnej wizualizacji. Aplikacja C działa jako bezstanowy silnik CLI, który przyjmuje topologię grafu, wykonuje intensywne obliczenia rozmieszczające wierzchołki i zwraca gotowe współrzędne. Została zaprojektowana z myślą o pełnej integracji z modułem GUI tworzonym w języku Java.

Dokumentacja ta łączy opisy funkcjonalne, implementacyjne oraz szczegółowe instrukcje pozwalające zespołowi docelowemu na sprawną integrację z silnikiem, bez konieczności analizowania kodu źródłowego w języku C.

2 Architektura Systemu i Implementacja

Aplikacja została napisana w standardzie C11 z wykorzystaniem kompilatora GCC. Architektura systemu ściśle przestrzega zasad *Clean Code*, *Minimal Global State* oraz *Defensive Programming*.

2.1 Struktura Modułowa

Projekt dzieli się na cztery główne warstwy:

- **core/** – Kontroler sterujący przepływem programu. Przetwarza flagi CLI (przy użyciu biblioteki `getopt.h`) i agreguje działanie całego programu.
- **io/** – Moduł wejścia/wyjścia. Obejmuje obsługę formatu tekstowego (wczytywanie grafu) oraz eksport do formatów tekstowych i binarnych.
- **algorithms/** – Główny rdzeń obliczeniowy zawierający logikę algorytmów Fruchtermana-Reingolda oraz Tutte’a.
- **utils/** – Zbiór pomocniczych funkcji matematycznych, m.in. implementacja eliminacji Gaussa dla algorytmu barycentrycznego.

2.2 Kluczowe Struktury Danych

Stan grafu przechowywany jest w dynamicznie alokowanych strukturach, zarządzanych przez strukturę `Graph`.

```
typedef struct {
    int id;
    double x, y;
} Node;

typedef struct {
    char name[MAX_NAME_LEN];
    int n_1, n_2;
    double weight;
} Edge;

typedef struct {
    Node *nodes;
    int node_count;
    int node_cap;
    Edge *edges;
    int edge_count;
    int edge_cap;
} Graph;
```

Listing 1: Uproszczony model struktur danych

2.3 Specyfikacja Wewnętrznego API (Wybrane funkcje)

Interfejs zarządzania grafem (zdefiniowany w `src_c/io/graph.h`) oferuje m.in. następujące metody:

- `Graph* create_graph(void)` – Inicjalizacja pustej struktury. Zwraca wskaźnik lub `NULL` w przypadku błędu alokacji.
- `int graph_load_text(Graph *g, const char *path)` – Wczytuje listę krawędzi. Zwraca kod błędu w przypadku problemów z plikiem/formatem.
- `int graph_save_binary(const Graph *g, const char *path)` – Eksportuje wynik działania algorytmów do binarnego pliku wymiany.
- `void free_graph(Graph *g)` – Zwalnia wszystkie zaalokowane zasoby, zapobiegając wyciekom pamięci.

3 Instrukcja Użytkownika (CLI)

Silnik C jest w pełni sterowany z poziomu wiersza poleceń. Pozwala to na łatwe wywołanie go z poziomu Javy.

3.1 Argumenty Uruchomieniowe

Program obsługuje następujące flagi:

Flaga	Wymagane	Opis
-i <path>	TAK	Ścieżka do pliku wejściowego (tekstowego) z opisem grafu.
-o <path>	NIE	Plik wyjściowy ze współrzędnymi. Domyślnie wynik wypisywany jest na konsolę (stdout).
-a <alg>	NIE	Wybór algorytmu rozmieszczania: fr (Fruchterman-Reingold - domyślny) lub tutte (Tutte).
-n <int>	NIE	Liczba iteracji dla algorytmu FR (domyślnie: 200). Nie dotyczy algorytmu Tutte'a.
-b	NIE	Wymusza zapis do formatu binarnego.
-v	NIE	Tryb verbose. Wyświetla diagnostykę i postęp na stderr.
-h	NIE	Wyświetla ekran pomocy i kończy działanie programu.

Tabela 1: Argumenty CLI programu

Przykłady wywołania:

```
# Standardowe przeliczenie algorytmem FR i wyrzucenie do pliku
# tekstowego
./graph_layout -i wejscie.txt -o wynik.txt

# Obliczenia algorytmem Tutte'a i eksport binarny dla interfejsu
# Java
./graph_layout -i wejscie.txt -o wynik.bin -a tutte -b

# Uruchomienie z podgladem postępu (verbose)
./graph_layout -i wejscie.txt -a fr -n 500 -v
```

Listing 2: Przykłady użycia CLI

4 Format Danych

4.1 Plik Wejściowy (Tekstowy)

Format wejściowy stanowi płaską listę krawędzi oddzielonych znakami białymi. Każda krawędź posiada nazwę, identyfikatory węzłów, które łączy oraz wagę. Format jednej linii: <nazwa_krawedzi> <wierzcholek_A> <wierzcholek_B> <waga_krawedzi>

Przykład pliku input.txt:

```
E1 1 2 1.0
E2 2 3 1.5
E3 3 1 1.0
E4 3 4 2.0
```

4.2 Plik Wyjściowy (Tekstowy)

Jeśli nie użyto flagi `-b`, program wygeneruje plik tekstowy. Zawiera on wyliczone pozycje (x, y) unikalnych wierzchołków z dokładnością do 6 cyfr po przecinku.

Format jednej linii: `<wierzcholek> <x> <y>`

Przykład pliku `output.txt` (dla powyższego wejścia):

```
1 0.000000 0.000000
2 1.000000 0.000000
3 0.500000 0.866000
4 0.500000 1.866767
```

5 Specyfikacja Mostka Binarnego (C-Java)

W celu zapewnienia wysoce wydajnej wymiany danych między rdzeniem obliczeniowym C a interfejsem graficznym Java, głównym formatem produkcyjnym (flaga `-b`) jest zoptymalizowany plik binarny.

5.1 Struktura Rekordu Binarnego

Każdy węzeł grafu zapisywany jest sekwencyjnie. Nie występują tu żadne nagłówki plików. Rozmiar pojedynczego rekordu wynosi dokładnie **20 bajtów**:

- `int id` (4 bajty) – Unikalny identyfikator węzła.
- `double x` (8 bajtów) – Współrzędna X zapisana w standardzie IEEE 754.
- `double y` (8 bajtów) – Współrzędna Y zapisana w standardzie IEEE 754.

Spójność danych: Całkowity rozmiar pliku wyjściowego zawsze będzie równy $N \times 20$ bajtów, gdzie N to liczba unikalnych wierzchołków w przetworzonym grafie.

6 Zaimplementowane Algorytmy Rozmieszczania

System udostępnia dwa niezależne mechanizmy wyznaczania współrzędnych. Aplikacja Java powinna umożliwić użytkownikowi przełączanie między nimi (flaga `-a`).

6.1 Algorytm Fruchtermana-Reingolda (Podejście siłowe, -a fr)

Uniwersalny algorytm traktujący graf jak system fizyczny.

- **Zasada działania:** Krawędzie grafu traktowane są jak sprężyny przyciągające wierzchołki, natomiast same wierzchołki działają na siebie jak naładowane elektrycznie cząsteczki (siły odpychania).
- **Cechy szczególne:** Obliczenia wykorzystują wagi krawędzi (wpływają na siłę przyciągania). Posiada mechanizm chłodzenia zapewniający stabilizację w kolejnych iteracjach. System wymusza ramkę, zapobiegając ucieczce węzłów w nieskończoność.
- **Zastosowanie:** Radzi sobie z grafami niespójnymi oraz nieplanarnymi. Idealny dla większości ogólnych przypadków użycia.

6.2 Algorytm Tutte'a (Twierdzenie o sprężynach, -a wszystkie)

Algorytm barycentryczny dedykowany grafom planarnym, gwarantujący brak przecięć krawędzi.

- **Zasada działania:** Znajduje cykl w grafie, a następnie "przypina" go do obwodu geometrycznego wielokąta wypukłego. Pozycje pozostałych (wewnętrznych) wierzchołków wyznaczane są poprzez rozwiązywanie układu równań liniowych (metoda Gaussa), tak aby każdy wierzchołek znalazł się w środku ciężkości swoich sąsiadów.
- **Mechanizmy obronne:** Program automatycznie używa algorytmu DFS do znalezienia zewnętrznego cyklu. Dodatkowo wykorzystuje BFS do walidacji spójności grafu – **warunek konieczny**.
- **Zastosowanie:** Służy wyłącznie do grafów spójnych i planarnych (oraz posiadających cykl). W przypadku podania grafu niespójnego program zwróci kod błędu.

7 Sytuacje Wyjątkowe i Obsługa Błędów

Program stosuje rygorystyczne podejście defensywne. Zamiast powodować awarię (segmentation fault), w sytuacjach brzegowych zwalnia zasoby i zwraca odpowiedni kod zakończenia.

Kod	Typ Błędu	Przykłady / Opis
0	SUKCES	Wykonanie poprawne. Wyniki zapisane do pliku.
1	BŁĄD PLIKU (ERR_FILE)	Plik wejściowy nie istnieje lub brak praw dostępu. Brak uprawnień do zapisu pliku wyjściowego.
2	BŁĄD FORMATU (ERR_FORMAT)	Nie zgodna liczba kolumn w pliku wejściowym (np. 2 zamiast 4). Znaki niebędące liczbami tam, gdzie oczekiwano double/int.
3	BŁĄD ALGORYTMU (ERR_ALGO)	Uruchomienie algorytmu Tutte'a dla grafu bez cyklu lub dla grafu nie-spójnego. Prowadziłoby to do dzielenia przez zero w macierzy.
4	BŁĄD ARGUMENTÓW (ERR_ARGS)	Brak flagi -i. Podanie nieznanego parametru flagi (np. -a xyz).

Tabela 2: Specyfikacja kodów powrotu silnika

8 System Testowy i Walidacja Jakości

Aby zapewnić stabilność i przewidywalność pracy programu C, zaimplementowano zautomatyzowane ramy testowe. Dodatkowo użyto narzędzia Valgrind do kontroli pamięci.

8.1 Środowisko i Metodologia

Wykorzystano skrypt `generator.c`, który masowo produkuje przypadki brzegowe i obciążeniowe. Test runner analizuje kody wyjścia i weryfikuje ich zgodność z założeniami. Przeprowadzono m.in.:

- Testy stabilności FR: Obciążenia grafami > 500 wierzchołków dla 1000 iteracji.
- Testy spójności Tutte'a: Wstrzykiwanie grafów losowych w celu sprawdzenia reakcji modułu weryfikacyjnego.
- Weryfikacja precyzji: Ekstremalne wielkości wejściowe (wagi rzędu 10^{-6} do 10^7).

8.2 Raport z Wykonania Zestawu Testowego

Poniżej znajduje się udokumentowany zrzut logu ze zautomatyzowanego runnera. Potwierdza on 100% skuteczność obsługi sytuacji wyjątkowych.

```
=====
ROZPOCZYNAM AUTOMATYCZNE TESTY SYSTEMU
=====

--- TEST: Obciazeniowy - 100 wezlow (Tutte) ---
[ZALICZONY] Oczekiwano: 0, Otrzymano: 0

--- TEST: Obciazeniowy - 500 wezlow (FR) ---
[ZALICZONY] Oczekiwano: 0, Otrzymano: 0

--- TEST: Krawedziowy - Zbyt maly graf (Tutte musi przerwac - Kod
3) ---
[ZALICZONY] Oczekiwano: 3, Otrzymano: 3

--- TEST: Krawedziowy - Ekstremalne wagi (Precyzja double) ---
[ZALICZONY] Oczekiwano: 0, Otrzymano: 0

--- TEST: Weryfikacja Valgrind (Brak wyciekow pamieci) ---
[ZALICZONY] Oczekiwano: 0, Otrzymano: 0

=====
WYNIK TESTOW: 5 / 5 ZALICZONYCH
WSZYSTKIE TESTY ZAKONCZONE SUKCESEM!
=====
```

Listing 3: Log z konsoli z weryfikacji automatycznej

8.3 Gwarancja braku wycieków (Valgrind)

Jak prezentuje log, w każdym przypadku poprawnym oraz błędnym program wywoływał procedurę `free_graph()` oraz odpowiednie sprzątanie zasobów plikowych `fclose()`. Raport pamięci Valgrind ("Clean memory report") potwierdza **zero utraconych bajtów**, co oznacza, że wielokrotne wywoływanie aplikacji w tle (proces po procesie) nie zdestabilizuje środowiska operacyjnego użytkownika.

9 Wnioski

Silnik obliczeniowy **GraphVisualizer (C Core)** osiągnął status ukończony (wersja produkcyjna). Spełnia wszystkie założenia specyfikacji funkcjonalnej i implementacyjnej.

Zaimplementowane algorytmy poprawnie rozmieszczają węzły grafów. Architektura programu oparta na bezstanowym CLI i formacie binarnym udostępnia wysoce wydajne, klarowne API pozwalające na bezproblemową budowę docelowego GUI w środowisku Java. Zabezpieczenia matematyczne zapobiegające awariom na niepoprawnych danych wejściowych czynią program odpornym na błędy, co udowodniono w udokumentowanych, bezbłędnych przebiegach testów automatycznych.